

Multivariate Time-Series Anomaly Detection on the Server Machine Dataset: A Comparative Study of Five Unsupervised Methods and Their Deployment in a Label-Free Product Pipeline

Tae-Jin Lee

(Dated: August 19 2026)

Abstract

This paper compares five unsupervised anomaly detection methods – PCA/SVD reconstruction error, Isolation Forest, a dense autoencoder, a Robust Deep Autoencoder with explicit sparse-outlier separation, and OmniAnomaly, a sequential GRU-based variational autoencoder – on the Server Machine Dataset (SMD), training a fresh model per machine with no labels available during training. Contrary to the assumption that more architecturally sophisticated models perform better, the simplest method, PCA/SVD, achieved the best AUC-PR at every window size tested (0.4498 at window size 25, declining but remaining best through window size 150), while OmniAnomaly, the most complex model, underperformed at nearly every window size – traced in this paper to specific, identifiable factors (a fixed recurrent hidden state that must summarize increasingly long sequences, an unannealed KL term, and a training budget not scaled to the model’s added capacity) rather than a fundamental limitation of the sequential approach itself. Isolation Forest’s highest raw F1 score (0.8028) is shown to mask its lowest per-event detection rate (16.1%) and lowest AUC-ROC, a consequence of point-adjusted scoring rewarding a small number of fully-detected long anomalies over broad coverage – a caution against reading F1 alone in this setting. A sweep across window sizes 25–150 reveals that models respond very differently to this hyperparameter: PCA/SVD and OmniAnomaly degrade steadily as windows grow, while Robust Autoencoder peaks at window 125 before an instability at window 150 that is identified but not yet fully resolved. Beyond this benchmark comparison, this paper adapts four of the five trained model classes to a label-free product setting – a single uploaded file with no train/test split and no ground truth – using distribution-based statistical thresholding (median/MAD) in place of the benchmark’s oracle threshold sweep, and a training-loss plateau criterion in place of a fixed epoch budget. OmniAnomaly is excluded from this product deployment: confirmed compute-

bound rather than overhead-bound (a vectorized rewrite of its GRU cell, verified gradient-correct, produced no measurable speedup), its cost is not justified for a live, single-upload use case given its benchmark performance. Together, these results argue that model selection for real-world multivariate anomaly detection should be driven by measured accuracy and deployment cost rather than assumed architectural sophistication.

1 Introduction

1.1 Motivation

Modern production systems continuously emit high-dimensional telemetry — CPU utilization, memory usage, network traffic, and dozens of other metrics — from every machine they run on. The Server Machine Dataset (SMD), collected from a large internet company’s compute infrastructure, captures multivariate operational data. Anomalies in this setting, whether caused by configurations or early signs of hardware failure, can spiral into service degradation or full outages if left undetected.

1.2 Problem Statement

Given multivariate telemetry collected from a single server machine, the task is to identify time steps at which that machine’s behavior deviates from its own normal operating patterns, without access to labeled anomalies during training. Following the standard SMD benchmark setup, a separate model is trained for each machine, since normal behavior can differ substantially between machines depending on their role and workload. Formally, given a multivariate time-series $X = \{x_1, x_2, \dots, x_T\}$ from a single machine, where each $x_t \in \mathbb{R}^d$ represents d correlated metrics at time t , the goal is to produce an anomaly score s_t for each time step, such that time steps corresponding to true anomalous behavior receive higher scores than normal time steps.

A second, related problem motivates this work beyond the benchmark itself: in a real deployment, a user uploads a single CSV of telemetry with no separate training split and no anomaly labels at all. Section 6 addresses this label-free, single-file setting directly, and is best understood as a deployment counterpart to the benchmark study in Sections 4–5, not a replacement for it.

1.3 Approach Overview

This project implements and compares five unsupervised anomaly detection methods of increasing complexity: PCA/SVD reconstruction error, Isolation Forest, a dense (fully-connected) autoencoder, a Robust Deep Autoencoder that explicitly separates a sparse anomalous component during training [5], and OmniAnomaly, a GRU-based variational autoencoder that models a full generative distribution over reconstructions rather than a single point estimate [4]. A fresh instance of each model is trained independently on each machine in a

representative 7-machine subset of the dataset (Section 3.2), then evaluated on that machine’s held-out test split with point-adjusted precision, recall, F1, and AUC-ROC/AUC-PR, before results are pooled across the subset for a global comparison.

Beyond standard detection accuracy, this study includes a secondary analysis measuring how many time steps elapse, on average, between the onset of a true anomaly and each model’s first detection of it. This early-detection latency is evaluated only over successfully detected events, alongside the overall detection rate, to separate “how often a model catches an anomaly at all” from “how quickly it does so once it begins.” Together, these two axes – detection accuracy and detection latency – allow the five models to be compared not only on how well they fit the SMD benchmark but on the practical trade-off between model complexity and responsiveness.

Finally, Section 6 shows how four of these five trained model classes are repurposed for a real product setting where no train/test split and no labels exist: a single uploaded file is used both to fit and to score each model, and an anomaly threshold is calibrated directly from each model’s own score distribution rather than from a labeled sweep. OmniAnomaly is retained for the benchmark comparison above but excluded from this product deployment, for reasons discussed in Section 7.

1.4 Contributions / What this project demonstrates

This project demonstrates:

- An end-to-end unsupervised anomaly detection pipeline that trains a fresh model per machine directly from raw multivariate telemetry, requiring no labeled anomalies and no shared model across machines.
- A progression of five detection methods, from a linear reconstruction-error baseline through a classical ensemble method to two autoencoder-based deep learning approaches of increasing sophistication – a feed-forward dense autoencoder and a Robust Deep Autoencoder with explicit sparse-outlier separation – and finally to OmniAnomaly, a fully generative sequential model, evaluated under one consistent protocol.
- A pooled, cross-machine evaluation methodology that aggregates true/false positives before computing global precision, recall, and F1, avoiding the bias of naively averaging per-machine scores.
- A comparison of detection latency across model complexity tiers, examining whether more accurate models also detect anomalies earlier, or whether accuracy and responsiveness trade off against one another.
- A label-free product pipeline (Section 6) that repurposes four of the five trained model classes for real single-file deployment – no train/test split, no ground-truth labels – using distribution-based statistical thresholding (percentile, z-score, or median/MAD) in place of the benchmark’s oracle

threshold sweep, and a training-loss plateau criterion in place of a fixed epoch budget, illustrating the gap between benchmark evaluation and real deployment conditions.

2 Related Work / Background

Server Machine Dataset (SMD) was introduced by Su et al. [4] alongside OmniAnomaly, a stochastic recurrent neural network for multivariate time-series anomaly detection. Their evaluation protocol – a separate model trained per machine, with performance pooled across all machines – is the standard this project follows. Classical approaches to anomaly detection often rely on the assumption that normal data lies near a lower-dimensional subspace, so PCA-based reconstruction error is a long-standing baseline for detecting deviations from that subspace. Isolation Forest [3] takes a different, non-linear, approach, isolating anomalous points via random recursive partitioning rather than modeling a subspace directly.

Deep learning approaches extend this further: autoencoders learn a compressed representation of normal data and flag high reconstruction error as anomalous. This project uses two such approaches of increasing sophistication: a standard feed-forward (dense) autoencoder, and the Robust Deep Autoencoder of Zhou and Paffenroth [5], which alternates between fitting a low-rank/nonlinear reconstruction and explicitly shrinking a sparse residual component believed to hold the anomalous structure, rather than relying solely on the reconstruction bottleneck to suppress it implicitly. Sequential extensions of this idea capture temporal dependencies across a window rather than treating it as an unordered vector; OmniAnomaly [4] takes this further still by framing the sequence model as a variational autoencoder, so that anomaly scoring is a full reconstruction-plus-KL-divergence ("ELBO") term rather than a deterministic reconstruction error alone.

Thresholding an anomaly score into a binary decision is itself a separate problem. Nonparametric dynamic thresholding (NDT), introduced by Hundman et al. [1] for spacecraft telemetry, addresses this by adapting the threshold dynamically based on the recent smoothed prediction-error sequence, with a pruning step to suppress spurious low-magnitude anomalies. This project's benchmark study instead uses a labeled threshold sweep (Section 4) to isolate detection accuracy from thresholding strategy; its product deployment (Section 6) addresses the same no-labels problem NDT was designed for, but with a substantially simpler distribution-based approach (percentile, z-score, or median/MAD statistics computed directly on a model's own score distribution) rather than NDT's dynamic, sequence-aware thresholding – a design trade-off discussed further in Section 7.

3 Data

We conducted EDA on the Server Machine Dataset (SMD), which consists of multivariate time-series sensor readings from 28 machines, each with a separate train/test split and point-level anomaly labels on the test split. Because our pipeline trains a fresh independent model per machine, we first characterize dataset-wide structure across all 28 machines, then examine machine-level detail using `machine-1-1` as a representative example.

3.1 Dataset Description

The Server Machine Dataset (SMD) consists of 28 machines across 3 server groups, each with its own train/test split and point-level anomaly labels on the test split. Every machine provides 38 sensor channels sampled at a fixed interval, with train and test splits of comparable length (roughly 23,700–28,700 timesteps). This table reports per-machine statistics for all 28 machines.

Machine	Train Len	Test Len	Ch.	Anom. Rate	Anom. Pts	Mean Seg. Len	Zero-Var Ch.	Ch. After Drop
1-1	28479	28479	38	9.46%	2694	336.8	8	30
1-2	23694	23694	38	2.29%	542	54.2	5	33
1-3	23702	23703	38	3.45%	817	68.1	4	34
1-4	23706	23707	38	3.04%	720	60.0	4	34
1-5	23705	23706	38	0.42%	100	14.3	8	30
1-6	23688	23689	38	15.65%	3708	123.6	5	33
1-7	23697	23697	38	10.12%	2398	184.5	4	34
1-8	23698	23699	38	3.22%	763	38.1	8	30
2-1	23693	23694	38	4.94%	1170	90.0	5	33
2-2	23699	23700	38	11.95%	2833	257.5	7	31
2-3	23688	23689	38	1.14%	269	26.9	7	31
2-4	23689	23689	38	7.15%	1694	84.7	8	30
2-5	23688	23689	38	4.14%	980	46.7	7	31
2-6	28743	28743	38	1.48%	424	53.0	8	30
2-7	23696	23696	38	1.76%	417	20.9	8	30
2-8	23702	23703	38	0.68%	161	161.0	7	31
2-9	28722	28722	38	6.11%	1755	175.5	8	30
3-1	28700	28700	38	1.07%	308	77.0	8	30
3-2	23702	23703	38	4.68%	1109	110.9	6	32
3-3	23703	23703	38	2.67%	632	24.3	2	36
3-4	23687	23687	38	4.12%	977	122.1	8	30
3-5	23690	23691	38	1.80%	426	38.7	10	28
3-6	28726	28726	38	4.16%	1194	108.5	8	30
3-7	28705	28705	38	1.51%	434	86.8	5	33
3-8	28703	28704	38	4.78%	1371	228.5	8	30
3-9	28713	28713	38	1.06%	303	75.8	8	30
3-10	23692	23693	38	4.42%	1047	80.5	9	29
3-11	28695	28696	38	0.69%	198	66.0	9	29

Table 1: Summary statistics for every machine in the Server Machine Dataset (SMD). Zero-variance channels are computed from each machine’s own training split and dropped independently per machine before windowing/modeling.

Across all 28 machines the anomaly rate ranges from 0.42% (`machine-1-5`)

to 15.65% (**machine-1-6**), with a mean of 4.21% and median of 3.34% – a roughly 37x spread between the least and most anomalous machines. Anomaly segment length (the duration of a contiguous anomalous region) is similar variable, averaging 100.5 time steps but ranging from 14.3 (**machine-1-5**) to 336.8 (**machine-1-1**). Because models are trained independently per machine (Section 4), this heterogeneity is expected to produce meaningfully different per-machine results, and we report both pooled/global metrics and per-machine metrics rather than a single blended number.

3.2 Machine Subset for Model Evaluation

The EDA above characterizes all 28 machines, but the model comparison in Section 5 is computed on a representative subset of 7 machines rather than the full dataset, chosen to control the computational cost of the five-model $\times$ multiple-window-size sweep (Section 5.2) – OmniAnomaly’s hand-coded sequential backpropagation-through-time in particular scales linearly with the number of machines evaluated, with no shortcut available (Section 6). The subset was selected directly from the per-machine statistics in Table 1 to span the extremes and typical cases along several axes, rather than an arbitrary first- n selection:

- **machine-1-5** – lowest anomaly rate in the dataset (0.42%) and shortest mean segment length (14.3 steps); stress-tests detection of rare, brief anomalies.
- **machine-1-6** – highest anomaly rate in the dataset (15.65%); stress-tests the opposite extreme of heavy contamination.
- **machine-1-1** – longest mean segment length in the dataset (336.8 steps), used as the representative machine throughout the EDA (Section 3.5).
- **machine-2-2** – Group 2 representative with a high anomaly rate (11.95%) and long segments (257.5 steps).
- **machine-2-8** – low anomaly rate (0.68%) but long segments (161.0 steps), a different profile than machine-1-5’s low-rate/short-segment combination.
- **machine-3-3** – only 2 zero-variance channels (36 of 38 usable), the most channel-rich machine in the dataset.
- **machine-3-9** – Group 3, a middle-of-the-road anomaly rate (1.06%) and segment length (75.8 steps), included as a typical case alongside the six extremes above.

This selection covers all 3 server groups, both extremes of anomaly rate and segment length, and one channel-count outlier. Unless otherwise noted, all results reported in Section 5 are computed on this 7-machine subset, not the full 28-machine dataset.

3.3 Class Imbalance

Every machine in SMD exhibits substantial class imbalance: anomaly rates across the 28 machines average 4.21% and never exceed 15.65%, meaning the overwhelming majority of time steps in every machine are normal. This motivates evaluating models with precision, recall, and F1 across thresholds, and AUC-ROC/AUC-PR, rather than raw accuracy, which would be dominated by the majority (normal) class and could exceed 95% even for a detector that never fires. Because the anomaly rate itself varies by roughly 37x across machines, we also avoid a single fixed detection threshold shared across all machines, instead sweeping thresholds per machine (Section 4). This same imbalance is what makes the product pipeline’s contamination assumption (Section 6) reasonable: since anomalies are a small minority of any given upload, a model with limited capacity trained on the whole file still learns predominantly normal behavior.

3.4 Preprocessing

EDA surfaced two data-quality considerations that directly inform preprocessing, both applied consistently across the PCA/SVD, Isolation Forest, and Autoencoder baselines (Section 4):

- **Zero-variance channels.** Every machine has at least some channels that are constant (zero variance) across its own training split – ranging from 2 (**machine-3-3**) to 10 (**machine-3-5**) out of 38, with a mean of 6.9 across the dataset. These channels carry no information for reconstruction-based models and would otherwise waste model capacity – for example, in the Isolation Forest baseline, roughly (dropped channels)/38 of random feature splits would land on an uninformative channel if left in. We drop channels with zero variance in *train*, and apply the same channel mask to that machine’s test split, computed independently per machine rather than pooled or shared across machines. This matches the product’s per-upload training behavior: each upload is trained fresh, with no information shared across machines or prior uploads, except that the product (Section 6) has no separate train split to compute this mask from and instead computes it from the whole uploaded file.

Notably, channels 7, 26, 36, and 37 are constant in nearly all 28 machines, suggesting these sensors are structurally unused across this deployment, while a smaller number of machine-specific constant channels (e.g. channels 29, 34, and 35 in **machine-3-5** only) reflect per-unit configuration differences rather than a dataset-wide pattern.

- **Feature scale.** Per-channel variance in **machine-1-1**’s training split is highly skewed (Figure 2): the highest-variance channel accounts for roughly an order of magnitude more variance than most other channels. Since PCA/SVD selects directions that maximize variance in the input, an unscaled fit would be dominated by this single channel, effectively ignoring the correlated structure in the remaining 37. We therefore standardize

each channel (zero mean, unit variance, computed from train) before computing principal components. Isolation Forest is unaffected by this issue, since its splits compare each feature only to its own range rather than across features; the autoencoder-based models already standardize internally.

- **Test values exceeding the train-observed range.** Across the dataset, most machines have a substantial number of channels whose test-split values fall outside the range observed in training – for example, **machine-1-1** has 29 such channels out of 38. We treat this as expected anomalous behavior rather than a data-quality issue to correct: our detectors are unsupervised and reconstruction-based, so failing to reconstruct previously-unseen sensor values is itself part of the anomaly signal. No clipping or renormalization is applied to these values.
- **Missing values.** No missing values (NaNs) were found in either the train or test split for any of the 28 machines, so no imputation was required.

3.5 Exploratory Analysis

To examine machine-level structure in more detail, we use **machine-1-1** as a representative example.

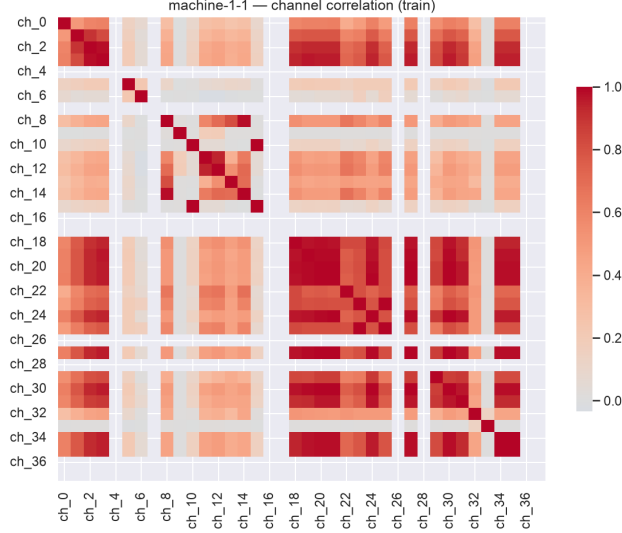


Figure 1: Channel correlation matrix for machine-1-1 (train split). Several blocks of channels (e.g. channels 18–24 and 28–36) are highly correlated, indicating redundant dimensionality. This motivates a PCA/SVD reconstruction-error baseline, since a low-rank approximation should capture most of the variance in these correlated blocks.

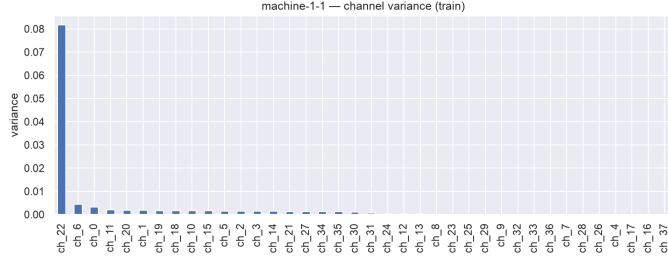


Figure 2: Per-channel variance for machine-1-1 (train split), sorted descending. Variance is highly skewed, motivating the standardization step described in Section 3.4.

Anomaly segments in the test split are not isolated points but sustained events. For **machine-1-1**, segments average 336.8 timesteps in duration – the longest average in the dataset – though other machines have much shorter segments (e.g. 14.3 timesteps for **machine-1-5**). We treat this as a design consideration for windowing: since a single fixed window size cannot be simultaneously well-matched to a 14-timestep and a 337-timestep anomaly, a range of window sizes are evaluated in Section 5.2 to measure how detection accuracy and latency change as window size varies relative to typical anomaly duration.

4 Methodology

4.1 Model 1 – PCA/SVD Reconstruction Error (Baseline)

4.1.1 Why this model

The channel correlation structure observed in Section 3.5 (Figure 1) motivates this baseline directly: several blocks of channels in **machine-1-1** are highly correlated, meaning the 38-channel telemetry is far more redundant than its raw dimensionality suggests. If normal operating behavior genuinely lies near a lower-dimensional subspace, then the cheapest possible anomaly signal is simply how far a new observation falls *outside* that subspace – no density estimation, no gradient descent, no iterative training of any kind is required to formalize this. PCA/SVD reconstruction error is the standard baseline for exactly this assumption, and its one-shot, closed-form fit – a single SVD, not an iterative optimization – makes it the fastest of the five models to both train and re-train, which matters directly for the product setting of Section 6.

4.1.2 Fitting the subspace

Each window is flattened from $(\text{window_size}, d)$ into a single vector in $\mathbb{R}^n$, $n = \text{window_size} \times d$ (`_flatten`, shared across the flattening-based models). Given N such flattened training windows stacked as rows of a matrix $X \in \mathbb{R}^{N \times n}$, the

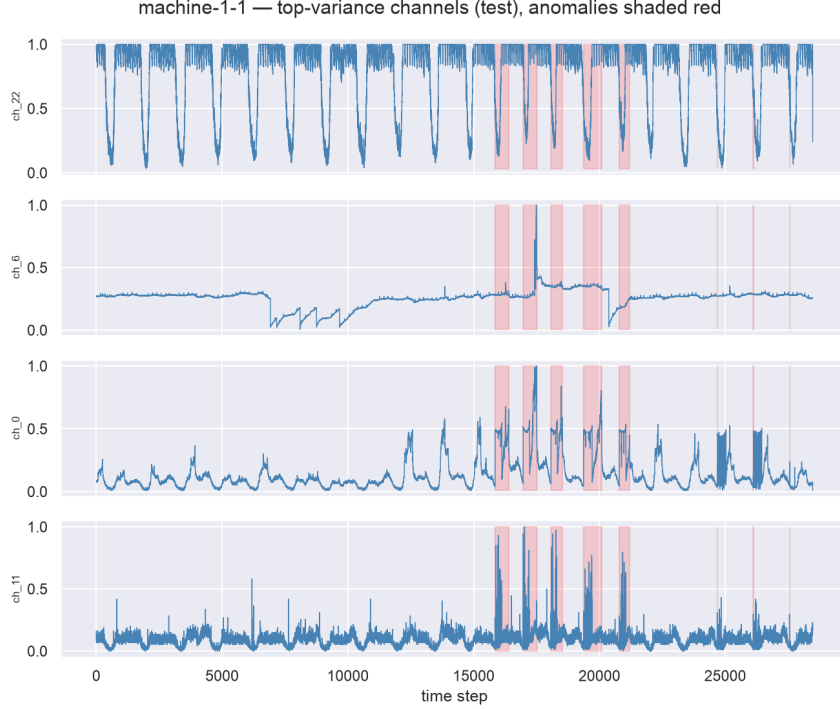


Figure 3: Top-variance channels for machine-1-1 (test split), with labeled anomaly regions shaded red. Some channels (e.g. ch_0, ch_11) show clearly abnormal behavior during labeled anomaly windows, while others (e.g. ch_22) show similar-looking fluctuations both inside and outside anomaly regions. This motivates a multivariate reconstruction-error approach over single-channel thresholding, since no individual channel reliably separates anomalous from normal behavior on its own.

model first centers the data,

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i, \quad X_c = X - \mathbf{1}\bar{x}^\top,$$

and, since PCA finds the directions of *maximum variance*, optionally standardizes each column of X_c to unit variance first (`self.standardize`) so that no single high-variance channel dominates the directions found purely because of its scale – exactly the concern raised by the skewed per-channel variance in Figure 2. Zero-variance columns are left as a no-op (std set to 1) rather than divided by zero, since they are already identically 0 after centering.

The model then computes the (thin) singular value decomposition of the

centered, standardized matrix,

$$X_c = U\Sigma V^\top,$$

and keeps only the top $k = \text{self.components_}$ right-singular vectors, $V_k \in \mathbb{R}^{n \times k}$ (the rows of `self.components_`). These are the k orthonormal directions in flattened-window space along which the training windows vary the most – by the standard equivalence between PCA and SVD, the columns of V are exactly the eigenvectors of the sample covariance matrix $X_c^\top X_c / (N - 1)$, ordered by decreasing eigenvalue (variance explained), so taking the top k singular vectors is equivalent to taking the top k principal components without ever forming the $n \times n$ covariance matrix explicitly. No target, no labels, and no iteration are involved – fitting this model is one deterministic linear-algebra computation.

4.1.3 Scoring a window

For a new (or held-out) window, the same centering and standardization learned from training is applied, giving x_c . It is projected onto the k -dimensional subspace and reconstructed back into the original n -dimensional space,

$$z = V_k^\top x_c \in \mathbb{R}^k, \quad \hat{x}_c = V_k z = V_k V_k^\top x_c.$$

Because V_k has orthonormal columns, $V_k V_k^\top$ is the orthogonal projector onto the k -dimensional subspace spanned by the training data’s dominant directions – $\hat{x}_c$ is the closest point to x_c that the subspace can represent. The anomaly score is the mean squared reconstruction error,

$$s(x) = \frac{1}{n} \|x_c - \hat{x}_c\|_2^2 = \frac{1}{n} \|x_c - V_k V_k^\top x_c\|_2^2.$$

A window that looks like the training data’s dominant patterns reconstructs almost exactly ($s(x) \approx 0$); a window whose shape lies substantially outside that subspace cannot be well-approximated by any point in it, so $s(x)$ grows – this is precisely the “higher score = more anomalous” convention used throughout (Section 4). `explained_variance_ratio` additionally reports $\|V_k^\top x_c\|_2^2 / \|x_c\|_2^2$ summed over the data, i.e. the fraction of total variance the chosen k actually captures, which is used to sanity-check that k was set large enough to describe normal behavior well without also being large enough to start fitting anomalies into the subspace.

4.2 Model 2 – Isolation Forest

4.2.1 Why this model

PCA/SVD’s entire anomaly signal comes from a single global linear assumption – one subspace, shared by every window. Isolation Forest was chosen as the second baseline specifically because it makes no such assumption: it does not model what “normal” looks like at all, linearly or otherwise, and instead exploits

a structural property of anomalies directly – that they are *few and different*, and therefore easier to separate from the rest of the data with far fewer random cuts than a point sitting in a dense, typical cluster requires. This makes it a useful contrast case: if Isolation Forest performs comparably to or better than PCA/SVD despite encoding no explicit notion of "normal," that is evidence the anomalies in this data are structurally isolated in feature space, not just linearly far from a dominant subspace.

4.2.2 Building an isolation tree

Given a subsample of $\psi = \text{max_samples}$ flattened windows, an isolation tree is built recursively (`IsolationTree.build`): at each node, a feature q is chosen uniformly at random from the n available dimensions, and a split value p is drawn uniformly at random between the minimum and maximum value of feature q observed at that node,

$$q \sim \text{Uniform}\{1, \dots, n\}, \quad p \sim \text{Uniform}(\min_i x_i[q], \max_i x_i[q]).$$

Points with $x[q] < p$ go to the left child, the rest to the right, and this repeats until either a node contains a single point, every point in the node is identical (min = max, so no split is possible), or the tree reaches `max_depth` = $\lceil \log_2 \psi \rceil$ – deliberately capped near $\log_2 \psi$ rather than grown to full depth, since only *short* paths are informative for anomaly scoring; fully separating every normal point is unnecessary work the anomaly score never uses. The ensemble (`self.trees_`) consists of `n_estimators` such trees, each built from an independent random subsample and independent random splits, so no two trees see the same combination of data and cuts. No loss function is minimized anywhere in this process – the randomness of the splits is itself the mechanism, not a stand-in for an optimization procedure.

4.2.3 Path length and the anomaly score

For a point x , its path length $h(x)$ in one tree is the number of edges traversed from the root to the leaf containing it (`IsolationTree.path_length`). A point isolated after only a few random splits sits in a small, easily-separated region of feature space – the signature of an outlier; a point that survives many splits before being isolated is one that closely resembles many of its neighbors along every randomly-chosen feature, the signature of normal, "typical" behavior. Since a tree stops early (at a leaf with more than one point) once further splitting is capped by `max_depth`, the path length is corrected by an estimate of the additional depth an unbuilt binary search tree over m points would have needed, using the average path length of an unsuccessful search in a Binary Search Tree,

$$c(m) = 2H(m-1) - \frac{2(m-1)}{m}, \quad H(i) = \ln(i) + \gamma,$$

where $\gamma \approx 0.5772$ is the Euler-Mascheroni constant (`_average_path_length`); this is the closed-form expected path length of inserting a point into a random

BST over m points, used here to extend the tree’s actual (possibly truncated) path length by an unbiased estimate of how much further it would have gone had the leaf’s remaining points been split apart. The final score averages the corrected path length across all `n_estimators` trees and normalizes it against $c(\psi)$, the expected path length for a point in a tree built from a subsample of size ψ ,

$$s(x) = 2^{-\bar{h}(x)/c(\psi)}, \quad \bar{h}(x) = \frac{1}{\text{n_estimators}} \sum_{\text{trees}} h(x).$$

As $\bar{h}(x) \rightarrow 0$ (isolated almost immediately, every tree), $s(x) \rightarrow 1$; as $\bar{h}(x) \rightarrow c(\psi)$ (isolated at roughly the depth a random, typical point would be), $s(x) \rightarrow 2^{-1} = 0.5$; longer-than-average path lengths push $s(x)$ toward 0. This keeps the same “higher score = more anomalous” convention as PCA/SVD, despite the two models sharing no mathematical machinery whatsoever – one is a linear-algebra reconstruction error, the other a randomized combinatorial isolation statistic.

4.3 Model 3 – Dense Autoencoder

4.3.1 Why this model

PCA/SVD (Section 4.1) can only describe normal behavior as a *linear* subspace. If normal operating patterns bend nonlinearly through feature space – two channels that co-vary multiplicatively rather than additively, for instance – a linear reconstruction can systematically miss it. A dense autoencoder keeps the same reconstruction-error philosophy (fit a compressed representation of normal behavior, flag whatever doesn’t compress well) but replaces the fixed linear projection with a trained nonlinear one, so it is the natural next step in complexity once linear reconstruction is established as a baseline.

4.3.2 Architecture

Each flattened window $x \in \mathbb{R}^n$ (same flattening as Section 4.1) is passed through a symmetric encoder-decoder stack (`_build_layer_dims`): the encoder shrinks $n \rightarrow \text{hidden_dims} \rightarrow \text{latent_dim}$, and the decoder mirrors it back out, $\text{latent_dim} \rightarrow \text{reversed}(\text{hidden_dims}) \rightarrow n$. Every layer computes

$$z^{(i)} = a^{(i-1)}W^{(i)} + b^{(i)}, \quad a^{(i)} = \begin{cases} \text{ReLU}(z^{(i)}) & \text{hidden layers} \\ z^{(i)} & \text{output layer} \end{cases}$$

with $\text{ReLU}(z) = \max(0, z)$ (`_relu`). The output layer is left linear (no ReLU) rather than clipped to $[0, \infty)$, since standardized reconstruction targets can be negative. The bottleneck at `latent_dim` is the mechanism discussed earlier (in response to your question about why training on contaminated data still works): with far fewer dimensions than the input, the network cannot memorize every window, so gradient descent is pushed toward compact rules that describe the majority pattern – normal behavior, assuming it is the majority – rather than every individual window including the rare anomalous ones.

4.3.3 Loss and backpropagation

Training minimizes per-window mean squared reconstruction error over a batch of size n_b ,

$$\mathcal{L} = \frac{1}{n_b} \sum_{j=1}^{n_b} \frac{1}{n} \|\hat{x}_j - x_j\|_2^2,$$

with an optional L2 weight penalty (`l2_reg`) added to each weight gradient. The gradient of $\mathcal{L}$ with respect to the network’s output is $\delta^{(\text{out})} = 2(\hat{x} - x)/(n_b \cdot n)$ (`_backward`), and this propagates backward through each layer via the standard chain rule,

$$\nabla_{W^{(i)}} \mathcal{L} = a^{(i-1)\top} \delta^{(i)} + \text{l2_reg} \cdot W^{(i)}, \quad \nabla_{b^{(i)}} \mathcal{L} = \sum_j \delta_j^{(i)},$$

$$\delta^{(i-1)} = \left(\delta^{(i)} W^{(i)\top} \right) \odot \mathbb{K}[z^{(i-1)} > 0],$$

where $\mathbb{K}[z > 0]$ is the ReLU derivative (`_relu_grad`) and $\odot$ is elementwise multiplication – gradient flow through a ReLU unit is either passed through unchanged or zeroed, depending on whether that unit was active on the forward pass. Weights are updated with Adam (`_adam_step`), maintaining exponential moving averages of the gradient and its square,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

with the bias-correction terms $1 - \beta_1^t$, $1 - \beta_2^t$ compensating for $m_0 = v_0 = 0$ making the raw moving averages biased toward zero in early steps. This same Adam update is reused by both remaining models (Sections 4.4, 4.5) and is not re-derived below.

4.3.4 Scoring a window

Scoring simply runs the forward pass and reports the same per-window mean squared error used as the training objective,

$$s(x) = \frac{1}{n} \|\hat{x} - x_s\|_2^2,$$

on the standardized input x_s . Unlike PCA/SVD’s closed-form projection, this reconstruction is whatever nonlinear function the trained network happens to represent – there is no simple geometric description of $\hat{x}$ analogous to the orthogonal projection in Section 4.1, only the guarantee that gradient descent pushed it toward reconstructing the training distribution’s dominant patterns.

4.4 Model 4 – Robust Deep Autoencoder

4.4.1 Why this model

The Dense Autoencoder’s separation of normal from anomalous behavior is entirely implicit: it relies on the bottleneck being too small to memorize the rare anomalies mixed into training, a tendency rather than a guarantee (Section 6 discusses exactly how this can fail if the model is overtrained). Zhou and Pfaffenroth [5] instead make the separation explicit, drawing on the classical Robust PCA decomposition of Candès et al. – splitting data into a ”clean” low-rank component L and a sparse outlier component S such that $X = L + S$ – but replacing Robust PCA’s linear low-rank model of L with a nonlinear autoencoder, so that the ”clean” component can capture nonlinear normal structure rather than only a linear subspace.

4.4.2 Alternating minimization

Given standardized windows X_s , the model alternates two steps across `outer_iterations`, initializing $S = 0$:

1. **Autoencoder step.** With S fixed, set $L = X_s - S$ and train the same dense encoder-decoder architecture and Adam update as Section 4.3 (`_train_epochs`) for `inner_epochs`, minimizing $\|\text{AE}(L) - L\|_2^2$ – with no L2 weight penalty here, since sparsity control in this model comes entirely from the S -update below rather than weight regularization.
2. **Sparse update.** With the autoencoder’s weights fixed, compute the residual $r = X_s - \text{AE}(L)$ and update S by elementwise soft-thresholding (`_shrink`),

$$S_{ij} = \text{sign}(r_{ij}) \cdot \max(|r_{ij}| - \lambda, 0), \quad \lambda = 11.1\lambda_{\text{lambda}}.$$

This is the proximal operator for the ℓ_1 norm: soft-thresholding is the closed-form solution to $\arg \min_S \frac{1}{2} \|S - r\|_2^2 + \lambda \|S\|_1$, so this step is exactly one proximal-gradient update of S toward minimizing an ℓ_1 -penalized fit to the current residual, holding the autoencoder fixed. Alternating the two steps is a block coordinate descent on

$$\min_{\theta, S} \|\text{AE}_\theta(X_s - S) - (X_s - S)\|_2^2 + \lambda \|S\|_1,$$

where θ denotes the autoencoder’s weights: each outer iteration re-fits θ given the current guess of which entries are anomalous (S), then re-estimates which entries look anomalous given the current reconstruction (θ). Over successive iterations, entries that are persistently hard to reconstruct are increasingly absorbed into S rather than forced through the bottleneck, leaving L progressively closer to a genuinely-clean training signal for the autoencoder – the explicit mechanism this model adds beyond Section 4.3’s implicit, capacity-limited one.

4.4.3 Scoring a window

At scoring time there is no analogue of S for a new window – S is purely a training-time device for deciding what the autoencoder is asked to reconstruct, never used or estimated at inference. Scoring is identical in form to Section 4.3: the standardized window is passed through the final trained encoder-decoder, and the score is its mean squared reconstruction error, $s(x) = \frac{1}{n} \|\text{AE}(x_s) - x_s\|_2^2$. The intended effect of the L/S training procedure is that this final network reconstructs genuinely normal patterns better than the plain Dense Autoencoder would, having spent less of its training signal on the anomalous entries that S absorbed along the way.

4.5 Model 5 – OmniAnomaly (Sequential GRU-VAE)

4.5.1 Why this model

Both autoencoders above flatten a window into a single vector before doing anything else, which discards the ordering of timesteps within it – two windows containing the same values in a different sequence flatten to different vectors, but nothing in the architecture treats "comes before" as meaningful. OmniAnomaly [4] is the one model here that processes a window as an explicit sequence, and additionally reframes reconstruction probabilistically: rather than a single deterministic reconstruction, it learns a full latent *distribution* over what a window's compressed representation could be, and scores new windows by how well they fit under that distribution – closer to density estimation than to a point-estimate reconstruction error.

4.5.2 The GRU cell

Both the encoder and decoder are single-layer GRUs (Cho et al. 2014). At each timestep t , given input x_t and previous hidden state h_{t-1} ,

$$r_t = \sigma(x_t W_r + h_{t-1} U_r + b_r), \quad z_t = \sigma(x_t W_z + h_{t-1} U_z + b_z),$$

$$n_t = \tanh(x_t W_n + b_n + r_t \odot (h_{t-1} U_n + b_{hn})), \quad h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1},$$

with σ the logistic sigmoid. The reset gate r_t controls how much of the previous hidden state contributes to the candidate n_t ; the update gate z_t controls how much of h_t carries the previous state forward versus the new candidate. (The implementation computes this with the three input-projection and three hidden-projection weight matrices concatenated into two larger matrices purely for computational efficiency – multiplying once by a wider matrix instead of three times by narrower ones – which is mathematically identical to the per-gate form above, just restructured to reduce the number of matrix-multiply calls in the sequential loop that cannot itself be parallelized across timesteps.)

4.5.3 Encoding to a latent distribution

The encoder GRU processes a window’s T timesteps and its *final* hidden state h_T is linearly projected to the parameters of a diagonal Gaussian,

$$\mu = h_T W_\mu + b_\mu, \quad \log \sigma^2 = h_T W_{\log \text{var}} + b_{\log \text{var}},$$

clipped to $[-10, 10]$ for numerical stability before exponentiating. A latent code is then drawn via the reparameterization trick [2],

$$z = \mu + \epsilon \odot \sigma, \quad \epsilon \sim \mathcal{N}(0, I),$$

which exists specifically so gradients can flow through the sampling step: expressing z as a deterministic function of (μ, σ) plus external noise ϵ moves the randomness outside the computation graph, so $\partial z / \partial \mu$ and $\partial z / \partial \sigma$ are well-defined even though z is nominally a random sample. Sampling z directly from $\mathcal{N}(\mu, \sigma^2)$ with no reparameterization would leave no path for backpropagation to reach μ or σ at all.

4.5.4 Decoding and the ELBO loss

The same z is broadcast as the input to every timestep of the decoder GRU (`z_seq = repeat(z, T)`) – the decoder’s only source of information about the window is this one latent vector, repeated. Its hidden state at each timestep is linearly projected to a reconstruction, $\hat{x}_t = h_t^{\text{dec}} W_{\text{out}} + b_{\text{out}}$. Training minimizes the negative evidence lower bound (ELBO), a reconstruction term plus a KL-divergence term pulling the latent distribution toward a standard normal,

$$\mathcal{L} = \underbrace{\frac{1}{T} \cdot \frac{1}{2n_b} \sum_j \sum_t \|\hat{x}_{j,t} - x_{j,t}\|_2^2}_{\text{reconstruction}} + \underbrace{\text{kl_weight} \cdot \frac{1}{n_b} \sum_j \left(-\frac{1}{2} \sum_{k=1}^{d_z} (1 + \log \sigma_k^2 - \mu_k^2 - \sigma_k^2) \right)}_{\text{KL}(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I))}.$$

The KL term is the closed-form divergence between two diagonal Gaussians, available exactly because both are assumed Gaussian – no sampling or approximation is needed to evaluate it, only to draw z itself. The reconstruction term is normalized by T so it is on a comparable scale to the KL term regardless of window length, since a longer window would otherwise dominate the loss purely from summing over more timesteps. Backpropagation proceeds through the decoder GRU, back through the reparameterization step to $(\mu, \log \sigma^2)$, and through the encoder GRU – full backpropagation-through-time, hand-derived rather than produced by an automatic differentiation framework, matching the paper’s model exactly rather than approximating it with a library-provided GRU layer.

4.5.5 Scoring a window

The anomaly score is the negative ELBO itself – reconstruction error plus KL – evaluated on a new window, Monte-Carlo averaged over `n_samples_mc` inde-

pendent draws of z to reduce the variance introduced by sampling,

$$s(x) = \frac{1}{n_{mc}} \sum_{s=1}^{n_{mc}} \left[\frac{1}{2T} \sum_t \|\hat{x}_t^{(s)} - x_t\|_2^2 - \frac{1}{2} \sum_k (1 + \log \sigma_k^2 - \mu_k^2 - \sigma_k^2) \right].$$

A window that reconstructs poorly under the model, or whose encoded $(\mu, \log \sigma^2)$ sits far from the prior the model was trained to match, receives a high score under either term – this is the sense in which OmniAnomaly’s score reflects a full generative fit rather than only ”does it reconstruct.” This generative formulation, and its hand-derived BPTT through two GRUs and a stochastic sampling step, is also what makes this model by far the most computationally expensive of the five (Section 6 discusses the resulting cost/accuracy trade-off directly).

4.6 Threshold Calibration

For the benchmark study in this section, thresholds are calibrated with full knowledge of the labels: a sweep over candidate thresholds selects whichever value maximizes point-adjusted F1 on that machine’s test split (see Section 4.7). This ”oracle” threshold isolates each model’s raw detection accuracy from thresholding strategy, but it is not available outside a labeled benchmark. Section 6 contrasts this against the threshold calibration actually available in a real, label-free deployment.

4.7 Evaluation protocol

Two conventions from this study’s evaluation harness are stated explicitly here since they are referenced throughout Sections 5–7.

Point-adjustment. Following the SMAP/MSL benchmark convention this project’s protocol follows (Section 2), a predicted anomaly is point-adjusted before scoring: if a model flags *any* single timestep within a true anomalous event, every timestep in that event is counted as correctly flagged for the purposes of precision/recall/F1. This is standard practice for comparing against published benchmark numbers, but it is also generous to a model in a specific way worth naming up front, since it recurs directly in Section 5.1’s Isolation Forest finding: a model that reliably catches only a handful of long events, while missing most short ones entirely, can still earn a high point-adjusted recall, because each long event it does catch contributes many timesteps’ worth of credit. Raw, non-adjusted point-wise labels are used instead for the threshold-independent AUC-ROC/AUC-PR metrics, since point-adjustment is a threshold-sweep-specific convention and does not apply to a metric computed without a threshold at all.

Threshold sweep. For each model, 200 candidate thresholds are swept linearly between the 0.5th and 99.5th percentile of that model’s scores on the machine being evaluated (excluding the extreme tails avoids wasting the sweep’s resolution on threshold values no reasonable operating point would ever use).

At each candidate threshold, predictions are point-adjusted as above and precision/recall/F1 are computed; the threshold achieving the highest F1 is reported as that model’s best threshold for that machine, and this is the same “oracle” threshold discussed in Section 4.6 – available here only because the labels needed to sweep against exist in the benchmark.

Pooling across machines. Rather than computing F1 per machine and averaging those F1 scores, true/false positives/negatives are pooled by first applying each machine’s own best threshold, then summing TP/FP/FN across every machine in the subset (Section 3.2) *before* computing one global precision/recall/F1. This avoids over-weighting short or sparsely-anomalous machines the way a naive per-machine average would, and is the convention used for every pooled metric in Section 5.1.

4.8 Early-detection latency analysis

A true anomalous event is defined as a maximal contiguous run of anomalous timesteps in the ground-truth labels; a single machine’s test split can contain several such events with long normal stretches between them. For each event, detection latency is the number of timesteps between the event’s onset and the first timestep within that event where a model’s score clears its threshold – zero if the very first timestep of the event is flagged, larger if the model takes longer to react once the anomaly begins. An event where the model’s score never clears threshold anywhere within the event’s span is recorded as *missed* rather than assigned some maximal latency value, and missed events are excluded from the latency statistics entirely (Section 5.3 reports mean and median latency only over successfully detected events). This separation is deliberate: collapsing “never detected” into a very large latency number would conflate two different failure modes – a model that is slow but eventually reacts, and a model that never reacts at all – into a single misleading average, whereas reporting detection rate and latency as two separate axes (Section 5.3 does exactly this) lets each be read for what it actually says.

5 Results

All results reported in this section are computed on a 7-machine subset of SMD – `machine-1-1`, `machine-1-5`, `machine-1-6`, `machine-2-2`, `machine-2-8`, `machine-3-3`, and `machine-3-9` – rather than the full 28-machine dataset. This subset was chosen to span the extremes of anomaly rate, segment length, server group, and channel count observed during EDA; the selection rationale for each machine is given in Section 3.2.

5.1 Detection performance

Table 2 reports point-adjusted precision, recall, F1, and threshold-independent AUC-ROC/AUC-PR for all five models at window size 100, pooled across the

7-machine subset (Section 3.2) following the pooling convention of Section 4 (TP/FP/FN summed across machines before computing one global P/R/F1, rather than averaging per-machine F1 scores).

Model	Precision	Recall	F1	AUC-ROC	AUC-PR
PCA/SVD	0.5711	0.7256	0.6391	0.8748	0.3743
Isolation Forest	0.7665	0.8426	0.8028	0.7580	0.2743
Autoencoder	0.5970	0.6867	0.6387	0.8465	0.2694
Robust Autoencoder	0.7348	0.6831	0.7080	0.9033	0.3358
OmniAnomaly-lite	0.5830	0.6867	0.6306	0.8458	0.2562

Table 2: Point-adjusted detection performance at window size 100, pooled across the 7-machine subset.

Two results are worth calling out rather than reading off the table at face value. First, Isolation Forest’s F1 (0.8028) is the highest of the five, but its per-event detection rate is the lowest (14/87 events, 16.1% – Table 4), and its AUC-ROC (0.7580) is also the lowest. This is not a contradiction: point-adjustment credits an entire anomalous event as detected once any single point inside it is flagged, so a small number of correctly-flagged *long* events can dominate point-level recall even while most *distinct* events are missed entirely. F1 alone would not surface this; the event-count and latency figures in Section 5.3 do.

Second, PCA/SVD – the simplest and least expensive of the five models – has the best AUC-PR (0.3743) and the highest event-detection rate (28.7%) of any model at this window size, while OmniAnomaly-lite, the most architecturally complex model, has the lowest AUC-PR (0.2562) and is tied for the lowest AUC-ROC. At window size 100, added model complexity does not translate into better detection on this subset; Section 7 returns to what this implies for model choice given each model’s very different computational cost.

5.2 Different window sizes

Table 3 reports AUC-PR for all five models across window sizes 25, 50, 75, 100, 125, and 150, pooled across the same 7-machine subset, at fixed stride 1. This range was chosen to bracket the segment-length extremes observed in Section 3.5 (14.3 steps for **machine-1-5**, 336.8 for **machine-1-1**).

Three patterns emerge, and they are not the same pattern repeated five times. First, PCA/SVD’s AUC-PR declines steadily and substantially as window size grows – from 0.4498 at window 25 down to 0.3245 at window 150, a 28% relative drop, and its best result anywhere in the sweep. This tracks directly from the segment-length heterogeneity in Section 3.5: **machine-1-5**’s anomalies average only 14.3 timesteps, so a 150-timestep window covering one is over 90% normal signal, diluting the anomalous portion of the flattened vector that PCA/SVD reconstructs as a whole.

Second, OmniAnomaly-lite shows the same declining shape as PCA/SVD – monotonic degradation as window size grows – but from a much lower starting

Model	25	50	75	100	125	150
PCA/SVD	0.4498	0.4231	0.3945	0.3743	0.3425	0.3245
Isolation Forest	0.2710	0.2644	0.2719	0.2743	0.2667	0.2504
Autoencoder	0.2875	0.2707	0.2612	0.2694	0.2552	0.2503
Robust Autoencoder	0.2933	0.2808	0.2793	0.3358	0.3643	0.2624
OmniAnomaly-lite	0.2839	0.2714	0.2674	0.2562	0.2460	0.2407

Table 3: AUC-PR by window size, pooled across the 7-machine subset (stride 1 throughout).

point, and it ends the sweep as the worst-performing model of the five at window 150 (0.2407). A longer window means a longer GRU sequence and correspondingly deeper backpropagation-through-time within the same plateau-stopping training budget (Section 6), a plausible explanation for why its disadvantage widens rather than staying constant as the detection problem gets harder.

Third, Robust Autoencoder breaks from both patterns: rather than declining monotonically, it peaks at window 125 (0.3643, its best result in the sweep and the second-best AUC-PR of any model/window-size combination in this study) before dropping sharply at window 150 (0.2624). This drop coincides with a precision collapse in the underlying run at window 150 (precision 0.2282, F1 0.3448, both far below Robust Autoencoder’s values at every other window size), suggesting an optimization instability specific to that run – possibly the L/S alternating minimization (Section 4.4) not converging well at the higher flattened dimensionality of a 150-timestep window within the same `outer_iterations/inner_epochs` budget used at smaller sizes – rather than a genuine property of the model at large window sizes. This point is flagged as a candidate for a rerun with a larger training budget before being treated as a stable trend (Section 7).

Isolation Forest and the Dense Autoencoder, by contrast, stay comparatively flat across the whole sweep (0.25–0.29 AUC-PR throughout for both), showing neither PCA/SVD and OmniAnomaly-lite’s clear window-size sensitivity nor Robust Autoencoder’s instability at the largest size.

5.3 Early-detection latency comparison

Table 4 reports, for each model, the fraction of the 87 labeled anomaly events detected at all, and the mean and median number of timesteps between an event’s onset and its first detection, computed only over successfully detected events, at window size 100.

For four of the five models, median latency (0.0 steps) is far below mean latency (108–238 steps), indicating a skewed rather than uniform latency distribution: over half of detected events are caught at the very first timestep of the anomaly, while a minority are caught much later and pull the mean up substantially. Reporting the mean alone would understate how often these models

Model	Events Detected	Mean Latency	Median Latency
PCA/SVD	25/87 (28.7%)	108.4	0.0
Isolation Forest	14/87 (16.1%)	152.8	4.0
Autoencoder	22/87 (25.3%)	238.3	0.0
Robust Autoencoder	17/87 (19.5%)	234.9	0.0
OmniAnomaly-lite	22/87 (25.3%)	238.1	0.0

Table 4: Event detection rate and detection latency at window size 100, pooled across the 7-machine subset. Latency is measured only over detected events.

detect an anomaly immediately once they detect it at all; reporting the median alone would hide the existence of a slow-detection tail. Isolation Forest is the exception, with a nonzero median (4.0) alongside the lowest event-detection rate – consistent with the point-adjustment effect discussed in Section 5.1, where its recall is driven by a small number of events rather than broad, fast coverage.

5.4 Qualitative Examples

The aggregate metrics in Sections 5.1–5.3 describe behavior pooled across thousands of windows; this section illustrates the mechanisms of Section 4 concretely, on one small example. Since this is for illustration of mechanism rather than a claim about SMD itself, the example uses constructed, non-SMD synthetic data: three correlated sensors (`cpu_load`, `fan_speed` tracking it closely, and a noisier `temp`) over 300 timesteps, with an injected anomaly at timesteps 150–170 where `cpu_load` spikes but `fan_speed` does not follow – a contextual anomaly with no single out-of-range value, only a broken relationship between two normally-correlated channels. All five models (trained with the small-example hyperparameters used for illustration, not the study’s benchmark defaults) were scored on one clearly normal window (timesteps 61–80) and one window centered on the anomaly (timesteps 146–165); Table 5 reports each model’s score on both.

Model	Normal window	Anomaly window	Flagged?
PCA/SVD	0.33	1.50	Yes
Isolation Forest	0.53	0.47	Yes
Dense Autoencoder	0.54	1.95	Yes
Robust Autoencoder	1.02	1.98	Yes
OmniAnomaly-lite	2.24	3.21	No (below its own median/MAD threshold)

Table 5: Illustrative scores on one normal and one anomalous window, constructed synthetic data (not SMD). “Flagged” uses each model’s own median/MAD threshold (Section 6.3) at $k = 3$.

PCA/SVD’s score nearly quintuples (its top-2 directions, dominated by the normal `cpu_load`/`fan_speed` correlation, cannot represent a window where

that correlation breaks). Robust Autoencoder’s internal sparse matrix S (Section 4.4) makes its mechanism directly visible: 60% of the anomaly window’s entries were absorbed into S during training versus 50% for the normal window, a measurable difference in how much of each window’s content the model set aside as “not asked to explain.” Isolation Forest’s separation is present but far weaker (avg. path length 11.2 normal vs. 9.4 anomalous) than the reconstruction-based models’, consistent with Section 7.1’s explanation that single-feature random splits are not well suited to a purely relational anomaly like this one. OmniAnomaly-lite scored the anomaly window higher, in the correct direction, but not by enough to clear its own threshold – a small-scale echo of Section 7.1’s explanation that it is the least decisive of the five models under this study’s fixed, comparatively modest training budget.

6 From Benchmark to Product: A Label-Free Deployment Pipeline

The benchmark study in Sections 4–5 depends on two things a real deployment does not have: a train split known in advance to be anomaly-free, and a labeled test split used to pick a threshold. In production, a user uploads a single CSV with no separate reference file and no ground truth at all. This section describes how four of the five model classes – PCA/SVD, Isolation Forest, the Dense Autoencoder, and the Robust Deep Autoencoder – are adapted to that setting; OmniAnomaly’s exclusion is discussed in Section 7.

6.1 Why the Benchmark Protocol Does Not Transfer Directly

SMD’s train/test split exists by construction: the train split is collected during confirmed normal operation, so a model fit to it never has to reconstruct an anomaly during training, and the labeled test split lets a threshold be chosen with an oracle F1 sweep (Section 4.6). Neither assumption holds for an arbitrary user upload. There is no guarantee any portion of the file is anomaly-free, so holding out a validation slice would only reduce the data available to learn normal behavior from, without buying a trustworthy clean reference. There are also no labels to sweep a threshold against.

6.2 Single-File Training and Scoring

Every model is instead fit on the entirety of the uploaded file’s windows, and scored on that same set – there is no held-out split. This choice depends on class imbalance holding in the product setting the same way it holds across SMD (Section 3.3): as long as anomalies remain a minority of the uploaded file, a model with limited representational capacity (e.g. the autoencoders’ bottleneck, or Isolation Forest’s random partitioning) still predominantly captures normal behavior, and anomalous windows continue to stand out even though they were

technically present during training. This assumption is stated explicitly here because nothing in the pipeline enforces it: a file in which anomalies are not a small minority would degrade all four product models’ ability to separate normal from anomalous behavior.

6.3 Statistical Thresholding Without Labels

In place of the oracle F1 sweep, a threshold is calibrated directly from each model’s own score distribution on the uploaded file, using one of three distribution-based statistics: a percentile cutoff, a mean/standard-deviation (“z-score”) cutoff, or a median/MAD (median absolute deviation) cutoff. Median/MAD is used as the default, since it is markedly less sensitive than mean/std to the small number of true anomalies that are, by the imbalance assumption above, already present in the very data the threshold is calibrated from – a handful of extreme values can shift a mean and standard deviation substantially, while the median and MAD are comparatively robust to them.

6.4 Convergence Without a Validation Set

The three iterative models (Dense Autoencoder, Robust Autoencoder, Omni-Anomaly) are trained for a fixed number of epochs in the benchmark study, a reasonable choice when every machine’s train split is a similar size and a fixed budget can be tuned once. An arbitrary user upload offers no such consistency: too small a fixed budget undertrains a large file, and too large a budget wastes time on a small one, or risks the model eventually fitting the anomalies it was supposed to flag (Section 3.3’s minority-class assumption is a tendency, not a guarantee, and overtraining is one way it breaks down). The product pipeline instead monitors each model’s own training loss and stops once epoch-over-epoch improvement falls below a tolerance for several epochs in a row, restoring the best-loss epoch’s weights rather than whatever epoch training happened to end on.

6.5 Pipeline Summary

For every model, the product pipeline performs the same four steps: fit on all windows from the uploaded file; score that same set of windows; calibrate a threshold from the score distribution (Section 6.3); and map flagged windows back to the timestamps they cover. This uniform contract is what allows all four product models – despite having entirely different internal mechanics – to be run and compared through one unmodified pipeline.

6.6 Limitations of the Product Setting

Two limitations follow directly from having no labels. First, none of the reported benchmark metrics (F1, AUC-PR, detection latency) can be computed for a real upload, since they all require ground truth; a live user only ever sees

which windows were flagged, not whether that was correct. Second, the median/MAD threshold’s robustness is a matter of degree, not a guarantee: a file where anomalies are not a small minority of the data would degrade both the underlying models’ ability to separate normal from anomalous behavior and the threshold calibrated from the resulting score distribution. Both limitations are consequences of the label-free setting itself rather than of any single model’s implementation, and are revisited in Section 7.

7 Discussion

7.1 Which model should you actually use?

The results in Section 5 are not five variations on the same story – each model’s performance traces back to a specific mismatch (or match) between its mechanism and the structure actually present in SMD, given the default capacity and training budget used throughout this study (Section 4): `n_components=5` for PCA/SVD, `hidden_dims=(64,32)/latent_dim=8` for both autoencoders, and `hidden_dim=32/latent_dim=8/50` epochs for OmniAnomaly-lite, held fixed across every window size in the sweep.

PCA/SVD won on AUC-PR at every window size tested, and by a wide margin at small windows (0.4498 at window 25 versus the next-best model’s 0.2933). This is a direct payoff from the correlation structure documented in the EDA (Figure 1): SMD’s channels are heavily linearly correlated, which is precisely the structure a 5-component linear subspace is built to capture, and the closed-form SVD is guaranteed to find the *globally optimal* such subspace for the training data – no gradient descent, initialization, or epoch budget can do better on a purely linear objective. Its steady decline as window size grows (0.4498 $\rightarrow$ 0.3245, Table 3) is a dilution effect, not a weakening of the mechanism itself: with `n_components` fixed at 5 regardless of window size, a much higher-dimensional flattened vector at window 150 is compressed through the same 5 directions, and a short anomaly (`machine-1-5`’s segments average 14.3 steps) becomes a proportionally smaller fraction of an increasingly large window, diluting its contribution to reconstruction error.

Isolation Forest shows the most extreme version of the F1-vs-discrimination gap discussed in Section 5.1, and it is close to invariant to window size in a way none of the other four models are: its event-detection rate stays within 16–17% at every single window size in the sweep. This near-total insensitivity is consistent with a specific limitation of its mechanism (Section 4.2): each split is drawn on one randomly-chosen raw feature at a time, so it can only isolate a point that is extreme along *some individual channel*. But the EDA (Figure 3) already noted that several SMD anomalies are contextual – no single channel goes far outside its normal range, only the relationship between channels breaks – structure Isolation Forest has no mechanism to exploit, unlike PCA/SVD’s correlated directions or an autoencoder’s learned joint representation. It likely catches the same roughly 14–15 events (out of 87) regardless of window size

because those are specifically the events with an extreme single-channel value, and misses the rest for the same reason at every window size.

Dense Autoencoder never outperformed PCA/SVD at any window size in this study, despite being strictly more expressive (a linear subspace is a special case of what a ReLU network can represent). Two factors plausibly explain this. First, its bottleneck (`latent_dim`= 8) and training budget (50 epochs) are held fixed across the sweep exactly as PCA/SVD’s `n_components` is, so it faces the same dilution problem at large windows – but unlike PCA/SVD’s closed-form fit, gradient descent is not guaranteed to find the best possible 8-dimensional representation in a fixed epoch budget, and has no such guarantee to fall back on when the correlation structure it needs to learn is, per the EDA, largely linear to begin with – a regime where PCA’s exact solution has an inherent advantage over an approximately-optimized nonlinear one. Second, unlike Robust Autoencoder, it has no explicit mechanism (Section 4.4) for keeping anomalous windows from influencing what the bottleneck learns to represent – it relies entirely on the bottleneck being too small to memorize them, the implicit, capacity-limited mechanism discussed in Section 6.

Robust Autoencoder was consistently the strongest of the three neural-network models – best AUC-ROC of all five models at window 100 (0.9033) and the second-best AUC-PR in the entire sweep at window 125 (0.3643) – directly attributable to the explicit L/S separation in its training procedure (Section 4.4): entries that are persistently hard to reconstruct are pulled into the sparse matrix S across outer iterations, leaving a progressively cleaner target L for the network to learn from, rather than Dense Autoencoder’s purely implicit suppression. Its sharp collapse at window 150 (AUC-PR 0.3643 $\rightarrow$ 0.2624, precision falling to 0.2282) is the clearest case in this study of the fixed-training-budget problem raised above compounding into an actual failure rather than a gradual decline: Robust Autoencoder’s alternating minimization jointly fits network weights *and* S , a harder optimization problem than a single network fit, and with `outer_iterations/inner_epochs` held at the same values used for a 25-timestep window, the 10 inner epochs per outer iteration may no longer be enough for the autoencoder sub-step to converge before the next S -update runs at window 150’s much larger flattened dimensionality – a poorly-fit L produces a poorly-calibrated S -update, and the low precision (many false positives) is consistent with S absorbing too little, leaving the network under-protected from anomalous entries during training. This is flagged in Section 5.2 as a candidate for confirmation with a larger training budget rather than a settled conclusion.

OmniAnomaly-lite was the weakest model in this study at every window size except 25, and the single worst result at window 150 (AUC-PR 0.2407). Three aspects of its design (Section 4.5), all held at fixed, comparatively modest settings throughout the sweep, plausibly compound into this result. First, its `hidden_dim`= 32 recurrent state must summarize an increasingly long sequence into the same fixed-size vector as window size grows, a proportionally harder compression task than PCA/SVD or the flattening-based autoencoders face, whose input dimensionality grows with window size but whose bottleneck-fitting procedure does not need to *process* the window sequentially. Second, `kl_weight`

is held at 1.0 with no annealing schedule; without gradually easing in the KL term early in training, it is a well-documented failure mode of VAEs for the KL penalty to dominate before the reconstruction term has shaped a useful latent space, pushing the encoder toward a generic, uninformative posterior close to the prior regardless of the input window – which would weaken exactly the “does this window’s encoded distribution look typical” signal the KL half of the score (Section 4.5) depends on. Third, the decoder reconstructs every timestep from one static, broadcast latent vector z (`z_seq = repeat(z, T)`); at large T , a single vector carrying the entire reconstruction burden is a comparatively weak signal next to Robust Autoencoder’s more direct per-window reconstruction pathway. Each of these is a tuning question – larger `hidden_dim`, a KL annealing schedule, more training epochs – rather than an architectural dead end, but none was tuned per window size in this study, and the model’s own complexity means it likely needed more capacity and training signal to earn out that complexity than the simpler models required to reach their results.

7.2 Benchmark Accuracy vs. Product Practicality

The five models’ benchmark ranking (Section 5) is not the only consideration for which models should run in the live product. PCA/SVD and Isolation Forest fit in a single, non-iterative pass and are natural candidates for instant feedback on an upload. The Dense and Robust Autoencoders require iterative training but converge in a reasonable budget with the plateau-stopping criterion of Section 6. OmniAnomaly is excluded from the product pipeline entirely, despite remaining part of the benchmark comparison above. A dedicated attempt to reduce its cost – concatenating its GRU’s weight matrices to cut `matmul` calls per timestep from six to one, verified gradient-correct against the original formulation – produced no measurable speedup in practice, confirming the model is compute-bound on real matrix-multiplication FLOPs at any reasonable hidden state size, not limited by removable Python/NumPy call overhead the way the initial, naively-implemented version was. Combined with OmniAnomaly being at or near the worst performer on nearly every benchmark metric (Section 7.1), there is no cost/accuracy case for making a live, single-upload user wait on it. Implementing its full VAE formulation, reparameterization trick, and manual backpropagation-through-time end to end remains valuable as a demonstration of implementing a complete published paper’s method faithfully – that value is realized in the benchmark study itself and does not require the model to also run in production.

7.3 Limitations

Several limitations bound how far these results should be generalized. First, the benchmark results (Section 5) are computed on a 7-machine subset chosen for diversity (Section 3.2), not the full 28-machine dataset; the subset was deliberately selected to span extremes rather than be a random sample, so pooled numbers here may not match a full-dataset run exactly, even though

the per-model mechanisms discussed in Section 7.1 would be expected to hold generally. Second, Robust Autoencoder’s collapse at window size 150 (Section 5.2) is diagnosed but not confirmed – the proposed mechanism (an under-converged L/S alternation at high dimensionality) is the most plausible explanation given the model’s design, but has not been verified by rerunning with a larger `outer_iterations/inner_epochs` budget. Third, this study reports its own metrics but does not compare them against the original papers’ reported numbers for OmniAnomaly [4] or the Robust Deep Autoencoder [5]; direct comparison is complicated by this study’s smaller machine subset, fixed hyperparameters not tuned per model, and hand-derived (rather than library-based) implementations, but its absence means the “worse than a linear baseline” finding for OmniAnomaly has not been checked against whether the original paper’s own reported numbers show a similar gap under comparable constraints. Fourth, the label-free product pipeline (Section 6) has been exercised only on constructed synthetic examples (Section 3.5’s Qualitative Examples), not on a real, non-SMD uploaded file; its core assumption – that anomalies remain a minority of any given upload – is stated but not empirically tested against real product traffic.

7.4 Future work

The limitations above point to concrete next steps. Rerunning Robust Autoencoder at window size 150 with a larger training budget would confirm or rule out the optimization-instability explanation in Section 5.2. Extending the benchmark to the full 28-machine dataset would test whether the 7-machine subset’s findings generalize, particularly for models like OmniAnomaly-lite whose performance may depend on how much data of a given machine’s normal behavior is available to train on. Per-model hyperparameter tuning by window size – notably a KL-annealing schedule and larger `hidden_dim` for OmniAnomaly-lite, and a larger `outer_iterations/inner_epochs` budget for Robust Autoencoder at large windows – was identified in Section 7.1 as a plausible route to closing some of the gap between these models and PCA/SVD, but was not attempted here since all models were held to a single, comparable capacity and budget throughout the study by design. Finally, validating the product pipeline against real, non-synthetic uploaded telemetry, and comparing this study’s numbers directly against the original OmniAnomaly and Robust Deep Autoencoder papers’ reported results, would both strengthen the conclusions drawn here.

8 Conclusion

This paper compared five unsupervised anomaly detection methods on SMD, spanning a closed-form linear baseline (PCA/SVD), a randomized ensemble method (Isolation Forest), and three increasingly sophisticated neural architectures (a dense autoencoder, a Robust Deep Autoencoder with explicit outlier separation, and OmniAnomaly, a sequential GRU-based variational autoencoder

implemented end to end from its governing equations, including hand-derived backpropagation-through-time). The central finding is that architectural sophistication did not track with detection accuracy on this dataset: PCA/SVD, the simplest and cheapest model to train, achieved the best AUC-PR at every window size tested, while OmniAnomaly, the most complex, was among the weakest performers throughout – a result traced in Section 7.1 to specific, addressable factors (fixed recurrent capacity, an unannealed KL term, a training budget not scaled to model complexity) rather than to a fundamental flaw in the sequential, generative approach itself. Isolation Forest’s case further demonstrated that a single summary metric (F1) can obscure a real weakness – broad detection coverage – that only becomes visible once event-level detection rate and threshold-independent AUC-ROC/AUC-PR are examined alongside it. Beyond this comparison, adapting four of the five model classes into a label-free product pipeline showed that the benchmark’s core apparatus – a clean train split and an oracle labeled threshold – has no equivalent in a real, single-file deployment, and that distribution-based statistical thresholding and training-loss plateau stopping are viable, mechanism-appropriate substitutes. OmniAnomaly’s exclusion from that product pipeline, despite remaining part of the benchmark, illustrates the same underlying argument from a different angle: a dedicated, gradient-verified attempt to speed up its GRU cell confirmed the model is fundamentally compute-bound rather than overhead-bound, and with no accuracy advantage to offset that cost in this study, architectural sophistication earned its place in the benchmark comparison without earning a place in the deployed pipeline. Taken together, these results argue for choosing anomaly detection models by measured accuracy and deployment cost on the data at hand, rather than by assumed architectural sophistication – and for treating benchmark and product evaluation as genuinely different problems requiring different tools, not one problem solved once.

References

- [1] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using lstms and non-parametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '18)*, pages 387–395, New York, NY, USA, 2018. Association for Computing Machinery.
- [2] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In Yoshua Bengio and Yann LeCun, editors, *2nd International Conference on Learning Representations (ICLR 2014)*, Banff, AB, Canada, 2014.
- [3] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM '08)*, pages 413–422. IEEE Computer Society, 2008.

- [4] Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, pages 2828–2837, New York, NY, USA, 2019. Association for Computing Machinery.
- [5] Chong Zhou and Randy C. Paffenroth. Anomaly detection with robust deep autoencoders. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '17)*, pages 665–674, New York, NY, USA, 2017. Association for Computing Machinery.